

Participate



ROAST MY DESIGN DECISIONS

Real-life trade-offs, success and failure

Steven Hillaert – solution architect





● Live

🗳 Multiple Choice

0 votes

□ Stop

What is your role?

Waiting for responses...

Results will appear here as people respond

Continuum of decisions

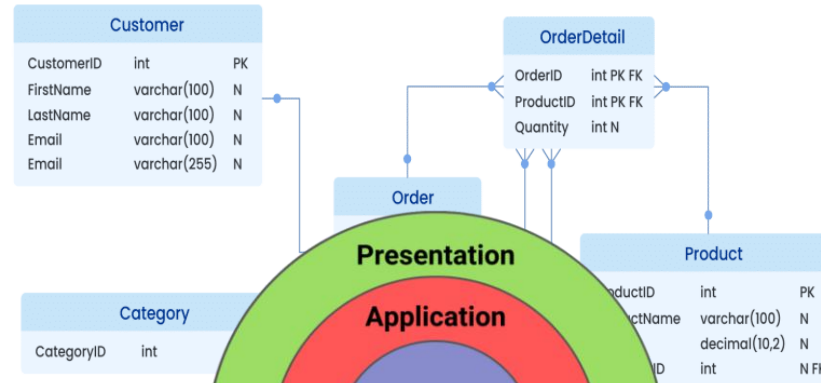
CODE
class/component

DESIGN
application/process

ARCHITECTURE
system

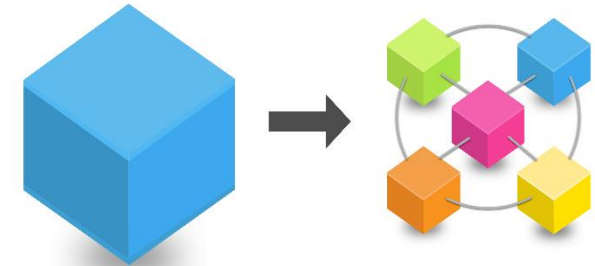
List vs Set vs Queue

ORM



Monolithic

Microservices



SOLID

CQRS

Easy to change

High impact



Don't follow my decision blindly

These decisions were made by balancing tradeoffs in a specific context:

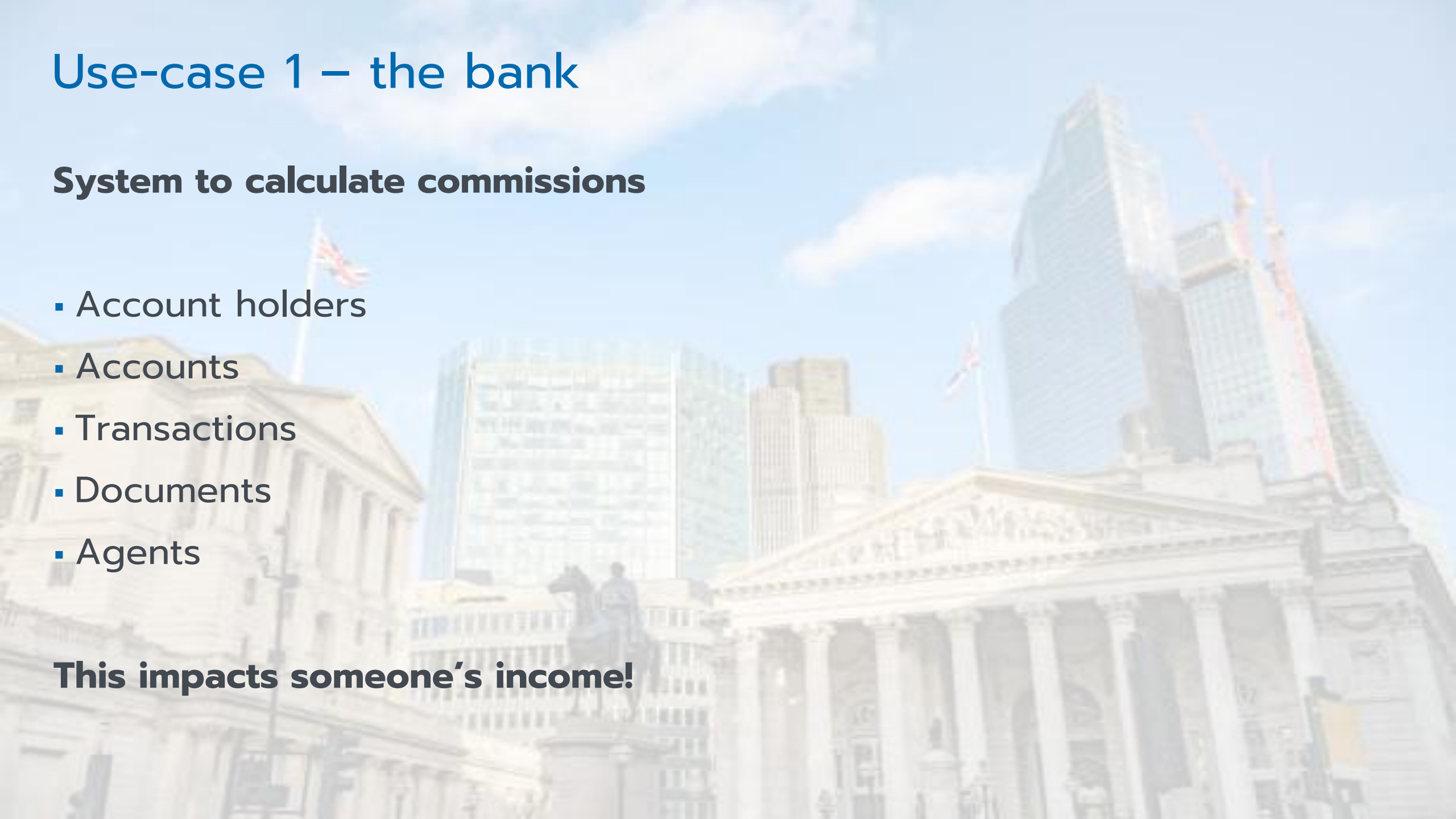
- Functional
- Non-functional
- Technical
- Strategic

Use-case 1 – the bank

System to calculate commissions

- Account holders
- Accounts
- Transactions
- Documents
- Agents

This impacts someone's income!



The bank

I designed a relational database without foreign keys.





● Closed

🗲 Rating Scale

0 votes

▶ Start

The bank - poll 1

Do you trust my database?

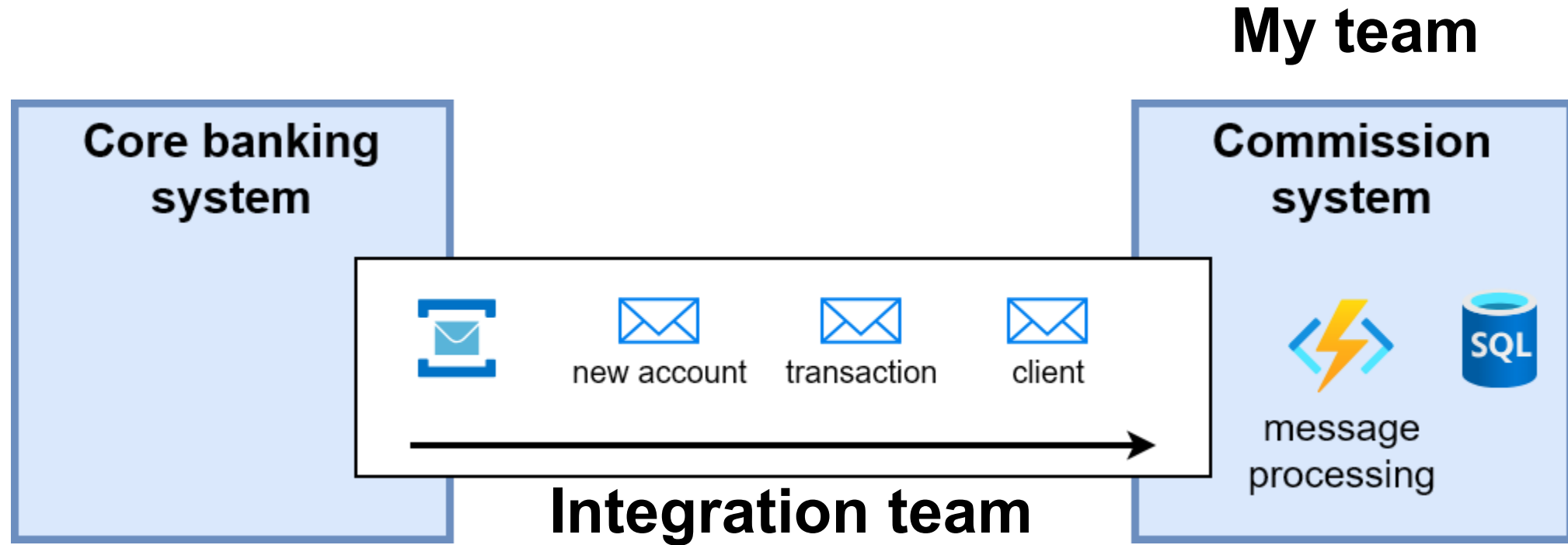
No responses received

This poll was closed without receiving any responses

The bank – project constraints

- Built as consulting service
- We deliver the application
- We only do 3rd level support
- Fixed time and budget
- They needed to validate that the latest data is in the commission system (timestamps)

The bank – system



The bank – standard challenges ⚠️

Standard challenges when using message brokers

- At-least-once delivery
- Out-of-order processing
- Poison messages / queues
- Monitoring
- Inbox pattern
- Idempotency
- Retries
- Built in Azure Service Bus 💰
- Tooling (NServiceBus) or custom 💰

Advice: idempotent messaging will save you from a lot of grief

The bank – integration challenges ⚠️

- Multiple messages for the same event
 - 2 create messages, identical data, different timestamp
- Different types of messages for the same event (but not always)
 - 1 update and 1 create message, different timestamp, any order
- Out-of-order publishing (very common)
 - Updates or deletes before creates
 - Transactions before accounts (in general: child entities before parents)

The bank – integration solutions 💡 (with FK)

- Multiple messages for the same event (2 creates)
 - ➔ Idempotency, keep the one with most recent timestamp
- Different types of messages for the same event (update + create)
 - ➔ Treat both as upsert, keep the one with most recent timestamp
- Out-of-order publishing
 - Updates before create ➔ Treat both as upsert, keep the one with most recent timestamp
 - Child entities before parents ➔ Retry until parent exists

The bank – current status

Still using foreign keys

- ✓ Fixed standard messaging challenges with idempotency
- ✓ Fixed quirky messages from producer with upsert & idempotency
- ✓ Avoided extra tooling/complexity
- ✓ Avoided extra costs
- ✗ Child entity messages before parent can be solved with retries
Requires monitoring, retries can still end up in poison queues, and verification with source system is difficult

The bank – socio-technical impact

Integration with source system isn't reliable.

Integration team and my team are external from different suppliers.

The original teams won't be there forever.

Support will be given by people who didn't develop the system.

The integration shouldn't create friction for the customer.

The bank – why do we use FK's?

FK's enforce referential integrity

The bank – why that breaks the system

Can't create child entities before parents

Can't create transaction before account

The bank – do we need it?

Do we need referential integrity?

(for this project)

The bank – do we need it?

Not really!

- We're not owner of that data
- Calculation behaves correct when parent entities would be missing
- Eventually consistent
- Messages can be processed as they come, no retries needed
- Monitoring inconsistencies is easier done from a database



● Closed

🗲 Rating Scale

0 votes

▶ Start

The bank - poll 2

How much trust in the current solution?

No responses received

This poll was closed without receiving any responses

The bank – core lessons

- Idempotency is non-negotiable in messaging systems
- Take a step back and look at the bigger picture
- Dare to challenge “defaults”
- Architecture is not about technical purity

Use case 2 – the inspection

Goal:

Standardize decision-making concerning damages (repair, insurance, invoicing...)

Users:

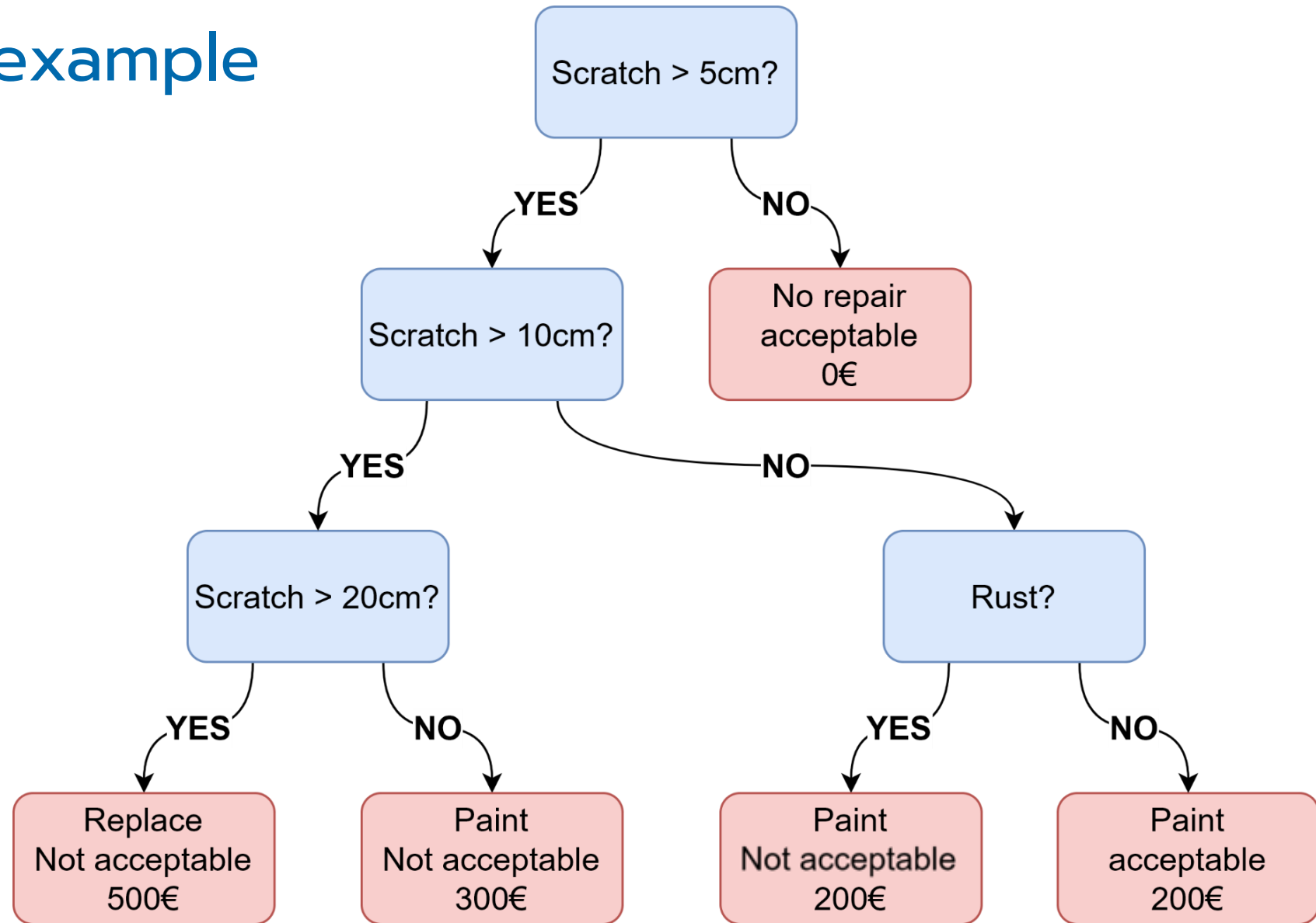
Professionals, fleet managers, drivers

Solution:

Decision tree with questions (per customer, damage,...)



The inspection - example



The inspection – the simplest solution

A tree structure where every node is a plain text yes/no question.

- Show the text (question) to the user
- User answers yes or no
- Show the next question
- Until you reach the end node

Workload: +/- 2 weeks

That's not what I did!



The inspection – the not so simple solution

- No dumb text-based questions
- Intelligent properties & comparators & type safe
- Builder & factory patterns
- Visitor pattern (traverse the tree in different ways)
- Supporting designer in the UI for configuration
- Multilingual

Workload: +/- 10 weeks



● Closed

🗲 Rating Scale

0 votes

▶ Start

The inspection - poll 1

Do you trust my overengineering?

No responses received

This poll was closed without receiving any responses

The inspection – a little philosophy

As an architect or tech lead, you're able to see the greater picture.

You're involved in more projects and stakeholders at strategic level.

It's not about predicting the future but the vision and strategy.

Will your decisions support or hinder it?

The inspection – what drove me

- **Changing** landscape of users
- **Data** is key, pure text doesn't create useful data
- Plain text questions don't **scale** very well when you need +500 trees
- Decisions need "metadata", everything needs to be **smart**
- **AI**/machine learning was being explored

The inspection – my decision

These things drove me to engineer it properly and completely, from the first version.

Too many future paths would need this. Even outside the inspection scope, as basis for a future rule engine.

We were agile in the capabilities offered but not in the core engineering.

Agile is not about creating the fastest and simplest solution!



● Closed

🗲 Rating Scale

0 votes

▶ Start

The inspection - poll 2

You trust my overengineering now?

No responses received

This poll was closed without receiving any responses

The inspection – a few weeks later

"We would like to have the actual values"

"Would this be compatible with AI?"

Starting with the simplest solution can be a risk for the future. (under-engineering)

We're conditioned to build on what we have.



Use case 3 – the microservices

- Small development team
- 1 monolithic system
- Supporting the core business
- Everything on-premise
- Growing team (x4)
- More customers and users
- Moving to the cloud
- European projects
- Expanding beyond the core business
- Cohesion between the new opportunities

The microservices – why?

- **Scalability** must increase to support the load
- **Reuse** must increase for shared data and services
- **Integration** must increase to create synergy
- New applications need more **autonomy**
- Growing teams need more **focus**
- Avoid single huge **deployments**
- We would do it **gradually**, we had a **plan**



● Closed

🗲 Rating Scale

0 votes

▶ Start

The microservices - poll 1

Were microservices the right choice?

No responses received

This poll was closed without receiving any responses

The microservices – the struggle

- Started with **foundation** services
- Built **core** business services on top
- Still using a **single database**
- Services started to **communicate** between each other (a lot)
- **Dependencies** got stronger
- Problems were fixed as **isolated** cases (distributed transactions)
- Teams were busy, **no governance**



● Closed

🗲 Rating Scale

0 votes

▶ Start

The microservices - poll 2

Are they still the best choice?

No responses received

This poll was closed without receiving any responses

The microservices – core lessons

- Choosing microservices was a good choice
- The implementation failed
- Boundaries & ownership are the real challenges
- Local optimizations can destroy global coherence
 - Get it working
 - Architecture drift
- Reuse is overrated

The goal of software design and architecture?

We write software to solve today's problems.

But we must design and architect solutions to continue delivering value in an uncertain, constantly changing future.

**Don't just look at upsides of a design or architecture.
Can you also live with the downsides?**

Questions and Feedback